PARTE 3

Protótipos, abstração e interface

Separando o que uma função oferece de como ela é implementada

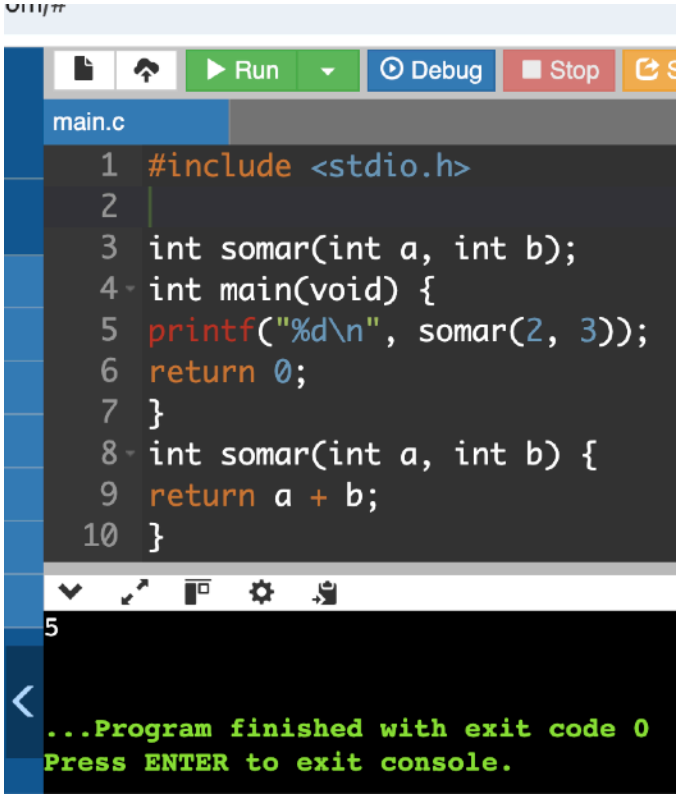
Aluno(a):	Antonio Maia de Jesus Chaves Neto	**Turma:**	Ciência da Computação_Noturno
Data:	27/09/2026	**Professor:**	Diego Abreu

Objetivos

- Distinguir declaração/protótipo, definição e chamada de função.
- Usar protótipos para permitir implementações após main.
- Compreender abstração e a ideia de interface versus implementação.

Atividade 1 - O problema da ordem

Considere um programa em que main chama somar antes de sua definição. O compilador precisa conhecer a assinatura da função antes do uso.



```
#include <stdio.h>

int somar(int a, int b);

int main(void) {
   printf("%d\n", somar(2, 3));
   return 0;
}

int somar(int a, int b) {
   return a + b;
}
```

1. **Declaração / protótipo vs. definição vs. chamada de função declaração (protótipo)**:

Informa ao compilador o nome da função, o tipo de retorno e os tipos dos parâmetros antes que ela seja usada. Ela termina obrigatoriamente com ponto e vírgula ; e não possui corpo (bloco de código). Vide a linha 3: int somar(int a, int b);

Por **definição**, ele contém a implementação real da função, incluindo a assinatura e o corpo delimitado por chaves {}.

Na chamada de função, ela executa a função passando os argumentos necessários e utilizando seu valor de retorno, se houver.

Exemplo na linha 5: somar(2, 3) dentro do *printf*.

2. Uso de protótipos para organização do código
O compilador C lê o código de cima para baixo. Se tentar chamar uma função antes de ela ser declarada ou definida, o compilador emitirá um erro ou aviso por não reconhecer a função.

Ao posicionar o protótipo (*int somar(int a, int b);*) no início do arquivo (linha 3), o compilador é notificado de que a função somar existe.

Isso permite que a função principal *main* (linhas 4–7) ficando no topo do arquivo, mantendo o fluxo do programa legível, enquanto as implementações completas das funções (linhas 8–10) podem ser organizadas logo após a *main.*

3. Abstração e a **relação interface vs. implementação**
Na Interface (O "O quê"): O protótipo atua como uma interface, ou contrato. Ele especifica o que a função precisa para ser executada (entradas: dois números inteiros) e o que ela produz (saída: um inteiro), sem revelar como o cálculo é feito.

A implementação (O “Como”), a definição contém os detalhes internos de funcionamento (neste caso, a operação de adição a + b).

A abstração permite que quem chama a função (como a main) utilize a funcionalidade *somar(2, 3)* sem precisar conhecer a lógica interna ou modificar o código da main caso a implementação interna da função venha a ser alterada futuramente.

> **Protótipo:** int somar(int a, int b); informa nome, tipo de retorno e tipos/quantidade de parâmetros.

Essa é a definição de um protótipo em linguagem C. Aonde:

int (no início): Informa o tipo de retorno (a função devolve um número inteiro).

somar: É o nome da função.

(int a, int b): Especifica os parâmetros — ou seja, que a função espera receber dois valores (quantidade) e que ambos devem ser do tipo int (tipo).

; (ponto e vírgula): Indica que é apenas uma declaração/assinatura, sem o corpo da função.

Essas informações é tudo o que o compilador precisa para verificar se você está chamando a função corretamente no main antes mesmo de ler a implementação dela.

Atividade 2 - Declaração, definição e chamada

Associe cada trecho ao conceito correto.

```
int somar(int a, int b);      /* ? */

int somar(int a, int b) {     /* ? */
   return a + b;
}

resultado = somar(5, 3);      /* ? */
```

- Declaração/protótipo: informa que a função existe.
- Definição/implementação: contém o corpo da função.
- Chamada: executa a função.

Para:

int somar(int a, int b); Neste caso, a Declaração/protótipo: informa ao compilador que a função existe, seu nome, tipo de retorno e os parâmetros que ela aceita, sem incluir o corpo.

int somar(int a, int b) { return a + b; } Neste caso, a Definição/implementação: contém o código-fonte/corpo da função, determinando o que ela faz quando é executada.

resultado = somar(5, 3); Neste caso, Chamada: invoca e executa a função passando os argumentos 5 e 3, atribuindo o retorno à variável **resultado.**

Atividade 3 - Reorganizando o main

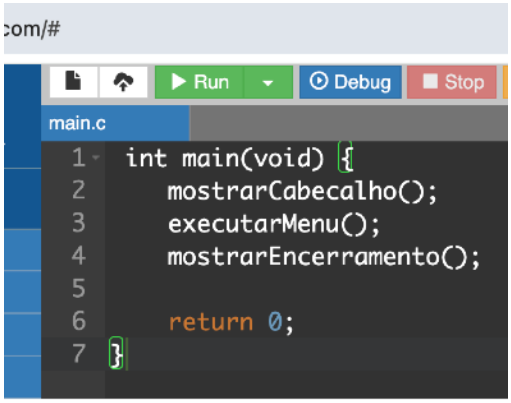
Transforme um main grande em uma visão de alto nível. Crie funções simples para que a estrutura principal se aproxime do exemplo:

```
int main(void) {
   mostrarCabecalho();
   executarMenu();
   mostrarEncerramento();
   return 0;
}
```

Mesmo sem ler os detalhes de cada função, deve ser possível compreender a intenção geral do programa.

A técnica descrita é o princípio da Abstração e da responsabilidade única. O objetivo é transformar uma função main poluída e cheia de detalhes de implementação em um script legível que funciona como um **sumário** ou **orquestrador** do programa.

Código final (Visão de alto nível)
Com a abstração aplicada, a função main passa a focar apenas no o que o programa faz, e não em como ele faz:

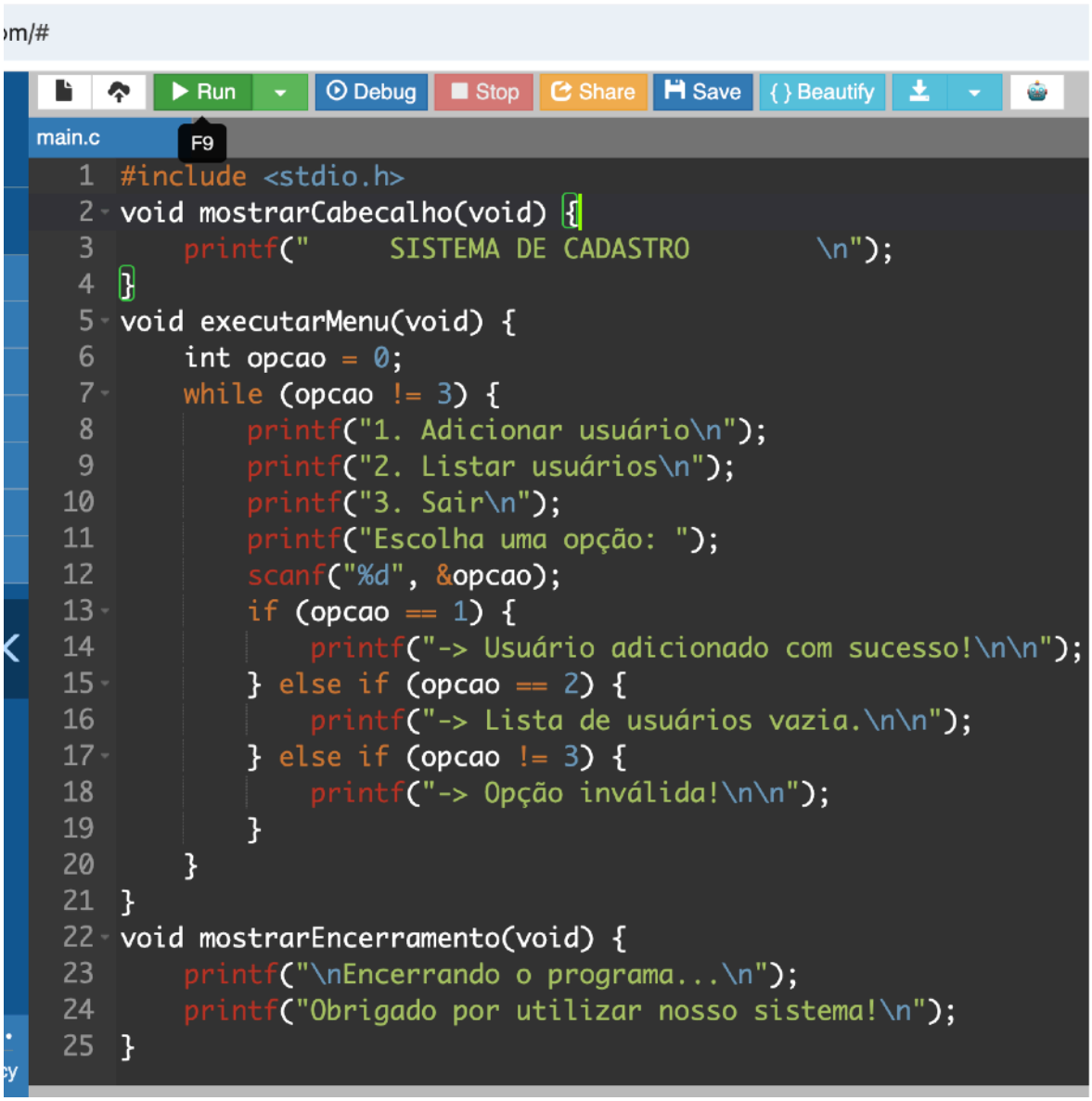


As vantagens desse tipo de abordagem:
A legibilidade para qualquer desenvolvedor entender o fluxo principal em segundos.

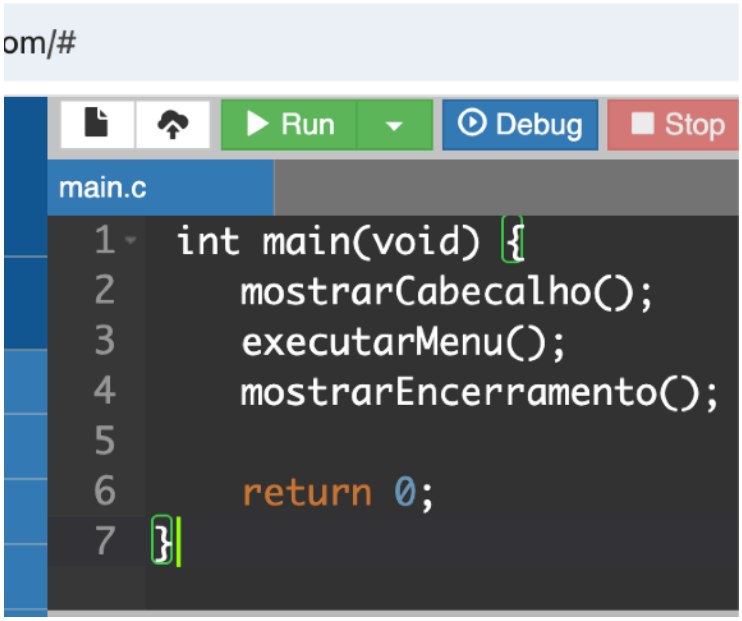
A manutenibilidade, no caso de precisar alterar o layout do cabeçalho, você altera apenas a função **mostrarCabecalho()**, sem risco de quebrar a lógica do menu.

A testabilidade deixa mais simples isolar e testar cada parte do sistema separadamente.

Do programa em linguagem C:



No final ficará com visão de alto nível:



Com a abstração aplicada, a função **main** passa a focar apenas no que o programa faz, e não em como ele faz.

As vantagens dessa abordagem são:
a) A legibilidade para qualquer desenvolvedor entender o fluxo principal em segundos;

b) A manutenibilidade, no caso de precisar alterar o layout do cabeçalho, você altera apenas a função mostrarCabecalho(), sem risco de quebrar a lógica do menu;

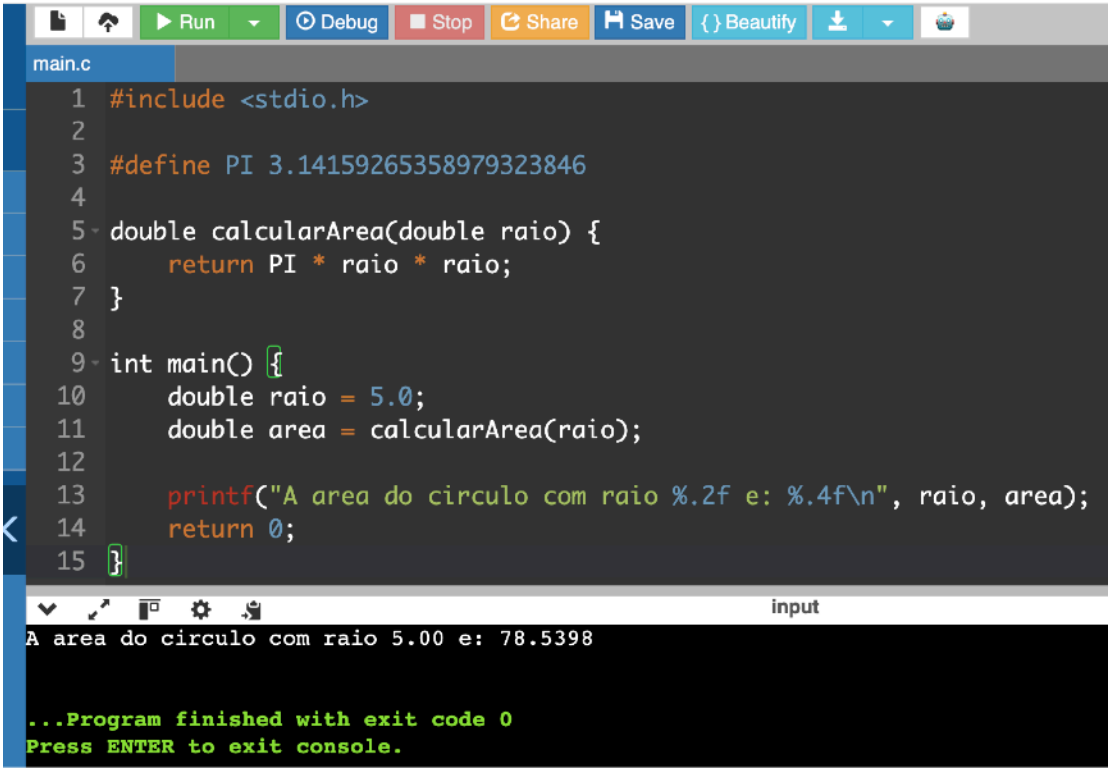
c) A testabilidade deixa mais simples isolar e testar cada parte do sistema separadamente.

> **Abstração:** trabalhar com uma representação mais simples e esconder detalhes que não são necessários naquele momento.

Atividade 4 - Interface versus implementação

```
double calcularArea(double raio);
```

A linha acima descreve uma interface simples: recebe um double e retorna um double.

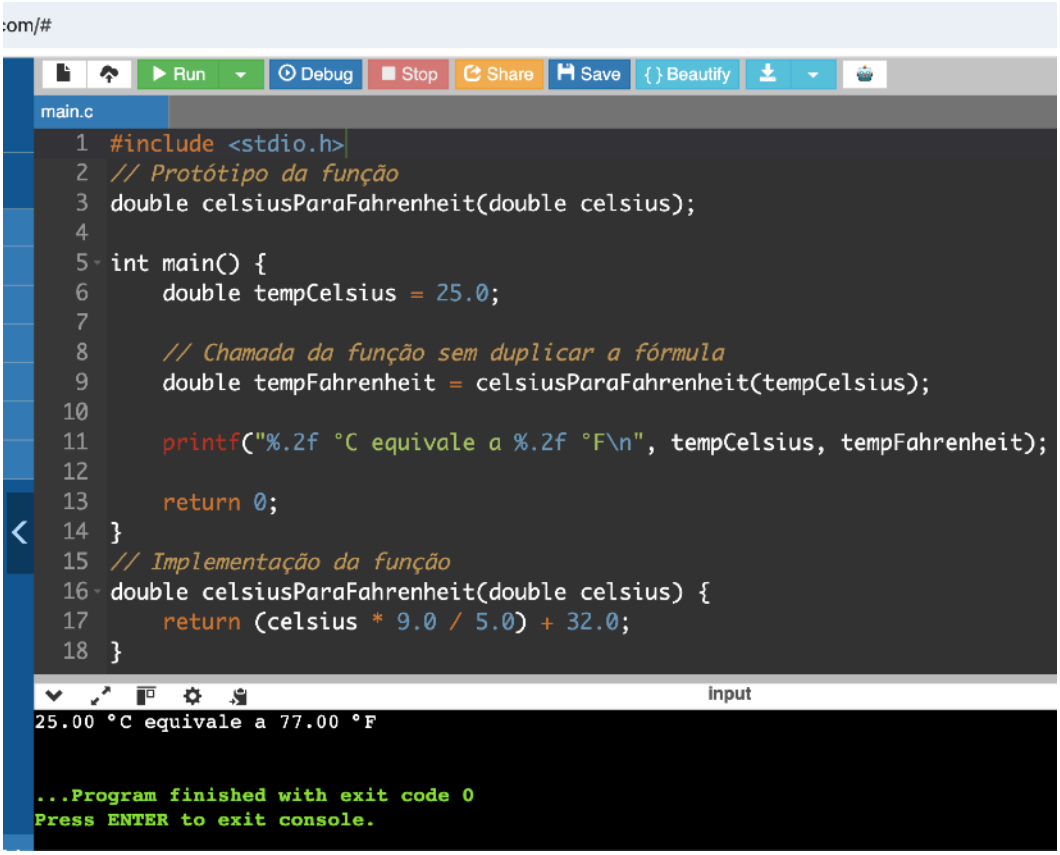



```
double calcularArea(double raio) {
   return 3.14159 * raio * raio;
}
```

A definição contém a implementação, isto é, os detalhes do cálculo.

Experimento

1. Escreva o protótipo de uma função celsiusParaFahrenheit.
2. Implemente a função após main.
3. Use-a em main sem duplicar a fórmula.



Atividade 5 - Reutilização e teste

```
double calcularDesconto(double valor, double percentual) {
    return valor * percentual / 100.0;
}
```

m/#

Run | Debug | Stop | Share | Save | { } Beautify

main.c

```c
#include <stdio.h>
// Declaração da função
double calcularDesconto(double valor, double percentual) {
    return valor * percentual / 100.0;
}
int main() {
    double valorOriginal, percentualDesconto;
    // Entrada de dados
    printf("Digite o valor do produto (R$): ");
    scanf("%lf", &valorOriginal);

    printf("Digite a porcentagem de desconto (%%): ");
    scanf("%lf", &percentualDesconto);
    // Chamada da função para calcular o desconto
    double desconto = calcularDesconto(valorOriginal, percentualDesconto);
    double valorFinal = valorOriginal - desconto;
    // Exibição dos resultados
    printf("\n--- Resultado ---\n");
    printf("Valor original: R$ %.2f\n", valorOriginal);
    printf("Desconto (%.1f%%): R$ %.2f\n", percentualDesconto, desconto);
    printf("Valor final a pagar: R$ %.2f\n", valorFinal);
    return 0;
}
```

```
Digite o valor do produto (R$): 100
Digite a porcentagem de desconto (%): 10

--- Resultado ---
Valor original: R$ 100.00
Desconto (10.0%): R$ 10.00
Valor final a pagar: R$ 90.00


...Program finished with exit code 0
Press ENTER to exit console.
```

```
Digite o valor do produto (R$): 500
Digite a porcentagem de desconto (%): 5

--- Resultado ---
Valor original: R$ 500.00
Desconto (5.0%): R$ 25.00
Valor final a pagar: R$ 475.00


...Program finished with exit code 0
Press ENTER to exit console.
```

Teste pelo menos: 100 com 10%, 500 com 5% e 250 com 20%. Depois altere a implementação para confirmar que as chamadas não precisam conhecer os detalhes internos.

Desafio - Arquitetura simples de conversões

Declare antes de main e implemente depois de main:

```
double celsiusParaFahrenheit(double celsius);
double fahrenheitParaCelsius(double fahrenheit);
double kmParaMilhas(double km);
void mostrarMenu(void);
```

O main deve mostrar o menu, ler a opção e chamar a função correspondente. Use switch para controlar as opções.

4. O protótipo é a mesma coisa que a implementação? Explique.

Resposta:

Não, o protótipo e a implementação não são a mesma coisa. Eles cumprem papéis completamente diferentes no desenvolvimento em C:

Protótipo (Declaração): Diz ao compilador o que a função é (seu nome, tipo de retorno e os parâmetros que recebe), sem mostrar o código interno. Funciona como um contrato.Implementação (Definição): Contém o código real entre chaves { ... } que diz como a função realiza a sua tarefa.O que um protótipo informa ao compilador?

5. O que um protótipo informa ao compilador?

Resposta:

O protótipo informa ao compilador a assinatura da função antes que ela seja chamada no código.

Especificamente, ele informa:

O nome da função: Para que o compilador a reconheça quando for chamada.

O tipo de retorno: Qual tipo de dado a função devolverá (ex: int, double, void).

A quantidade e o tipo dos parâmetros: Quais dados a função espera receber e em qual ordem, permitindo que o compilador verifique se você está chamando a função com o número e os

tipos de argumentos corretos, apontando erros de sintaxe antes mesmo de compilar o programa inteiro.

6. Como abstração ajuda a entender programas maiores?

Resposta:

A abstração é o conceito de esconder os detalhes internos complexos de um sistema e expor apenas o que é essencial para utilizá-lo.

Em programas maiores, ela ajuda das seguintes formas:

Redução da carga mental. Não é necessário ler nem entender milhares de linhas de código de uma biblioteca para usá-la; basta saber o que as funções fazem.

Modularidade: o programa é dividido em partes menores e independentes. Você pode focar em resolver um problema por vez.

Facilidade de manutenção: Se precisar alterar como uma função funciona internamente (a implementação), o restante do programa continuará funcionando, desde que o protótipo não mude.

7. O que é a “interface” de calcularArea(double raio)?

Resposta:

A interface de calcular Area(double raio) é a forma como o restante do programa se comunica com essa função, ou seja, o seu protótipo/cabeçalho.

Neste caso, a interface define:

Com o nome **calcularAreaEntrada** esperada, ou seja, um valor do tipo double que representa o raio.

Esta unitização poderá ser em qualquer parte do programa sabe que, para calcular a área, basta fornecer esse número, sem precisar saber se a função usa a fórmula de π simplificada, uma constante de precisão ou uma biblioteca matemática interna para realizar o cálculo.

Entrega da parte

- Programa usando protótipo de somar().
- Programa de conversões com protótipos antes de main e implementações depois.
- Respostas conceituais.